Dissertation Prospectus

Discovering new exploit techniques through generalizing known exploits

for

Yihui (Kyle) Zeng

Approved June 2025 by the
Graduate Supervisory Committee:

Tiffany Bao, Chair
Adam Doupé
David Brumley
Ruoyu Wang
Yan Shoshitaishvili

ARIZONA STATE UNIVERSITY

June 2025

TABLE OF CONTENTS

1 Introduction 1

Page

2 Playing For K(H)eaps 3

2.1 Introduction 3

2.2 Generalization - Kernel Heap Exploit Model 4

2.3 New Technique - Context Conservation & Combo Technique 5

2.4 Conclusion 6

3 Retspill 7

3.1 Introduction 7

3.2 Generalization - User-Controllable Data Injection 7

3.3 New Technique - RetSpill 8

3.4 Conclusion 9

4 ropbot 10

4.1 Introduction 10

4.2 Generalization - Gadget & rop_block 10

4.3 New Technique - ropbot 12

4.4 Conclusion 14

5 Plan For Defense 15

1 Introduction

Software vulnerabilities are dangerous. Malicious actors may take advantage of software vulnerabilities and cause catastrophic results, such as financial losses and exposure of personal information. In 2017, the WannaCry ransomware exploited a vulnerability in the Windows kernel, encrypting victims' files and demanding a ransom, which was estimated to have caused 6 billion in financial losses [9]. Started in 2019, LockBit ransomware group has been a threat to the security of the whole Internet. It gains access to victims' computers by exploiting multiple vulnerabilities, causing millions of financial losses [3].

The most critical step in those campaigns is the exploitation of the vulnerabilities, or known as exploits. These exploits start from an initial capability provided by the vulnerabilities (e.g. out-of-bound write), use different techniques to bypass protections in the target system, and eventually control the system completely.

Exploits are complicated and difficult to analyze, due to their nature of entanglement with low-level computer designs. As a result, they are understudied. There have been a few attempts to collect exploits, such as ExploitDB[1] and KernelCTF[2]. But there is still no comprehensive study of these exploits, showing what we can learn from these known exploits.

This dissertation aims to fill the gap. It investigates known software exploits, with the goal of discovering new exploitation techniques, revealing the weaknesses in computer software. More specifically, the thesis is that *understanding and generalizing known software exploits can lead to a better understanding of the target program (§ 2), identifying new attack surfaces (§ 3), and the discovery of new exploitation techniques (§ 2, § 3, § 4).*

This dissertation makes contributions to system security by showing 1) how to

generalize known exploits and 2) how to derive new exploitation techniques based on the generalization. More specifically, by analyzing Linux kernel heap exploits, we design a generic kernel heap exploit model, which led to the discovery of a new technique called Context Conservation (§ 2) that can stabilize exploit kernels. Our follow-up research indicates that its use is beyond stabilizing kernel heap exploits, it is also critical in stabilizing prefetch attacks. By inspecting how exploits inject their data into the Linux kernel, we identify a new attack vector, namely the kernel stack, that has not been explored before (§ 3). Further analysis revealed its critical nature to the security of the Linux kernel. By generalizing code reuse attack chain generation technique by generalizing the definition of a gadget, we design an architecture-agnostic code reuse attack chain generation technique and discover that the foundation of dynamic linking, Global Offset Table (GOT), can be abused as gadgets in code reuse attacks (§ 4). Overall, we argue that known exploits contain hidden knowledge that even the exploit developers may not know explicitly. By understanding and generalizing exploits, we can extract and formalize the knowledge and use it for hardening software security.

2 Playing For K(H)eaps

2.1 Introduction

Kernel heap exploit is a threat to Linux's security. In Google's Kernel CTF program, all its 70 public exploits demonstrate full system compromise through heap exploits.

It is known that kernel heap exploits are complex and unreliable. But what factors are behind the unreliability is understudied. Although it seems exploits being unreliable protects the kernel from attacks, it also impacts developers' ability to analyze and reproduce the vulnerabilities, thus delaying the fixes.

In this work, we aim to address this situation by studying and developing techniques that increase the reliability of kernel exploits, helping to properly demonstrate the impact and criticality of the underlying vulnerabilities. More specifically, we aim to address the *exploit reliability problem* – if a kernel heap vulnerability is exploitable, how to achieve a higher exploitation success rate and properly understand the chance of a successful attack.

To perform the study, we first exhaustively collected 17 exploits (the largest dataset at the time of the project, which was before KernelCTF launched). Then we interviewed 11 kernel exploitation experts to gain insights into exploitation techniques. Next, we ran the exploits with different exploitation techniques and recorded their success rates. Finally, based on the observation, we generalized the lifetime of heap exploits into a Kernel Heap Exploit Model and used the model to derive a new exploitation technique. This technique improves kernel exploit reliability by 14.87% and was proven to work beyond kernel heap exploits.

2.2 Generalization - Kernel Heap Exploit Model

The first step of our generalization is to exhaustively collect existing knowledge.

We collected 17 kernel heap exploits as the study target, which was the largest dataset at the time of the project. We then interviewed 11 kernel exploitation experts, from both vulnerability research companies and research institutes, gaining knowledge about exploit stabilization techniques and how they expect to work under different scenarios. In total, we collected five stabilization techniques, namely, **Defragmentation**, **Heap Grooming**, **Single-Thread Heap Spray**, **Multi-Process Heap Spray**, and **CPU Pinning**. Next, we stripped away existing stabilization techniques from the real-world exploits to form the baseline exploits and applied each technique to the baseline exploit as the experimental exploits. Some of the success rates of the exploits were misaligned with experts' expectations, which suggests the understudied and convoluted nature of the kernel exploit reliability problem.

We investigated all the exploits and techniques and identified four factors affecting exploit reliability, namely, *Unknown Heap Layout*, *Unwanted Task Migration*, *Unexpected Heap Usage*, and *Unpredictable Corruption Timing*.

By explicitly showing the lifetime of kernel exploits and pinpointing the critical phases (where exploits may fail) in the lifetime, we can understand how the four unreliability factors fail exploits in what critical stage. We named this model the Kernel Heap Exploit Model, which is shown in Figure 1.

This model helps further zoom in on the cause of failures of a kernel heap exploit and helps instruct the development of new stabilization techniques.

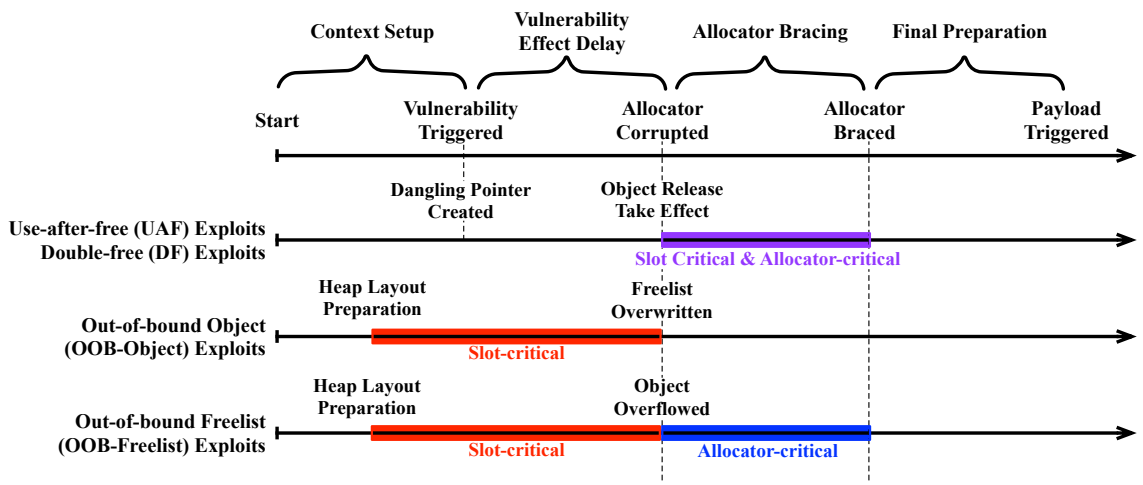

Figure 1: The kernel heap exploit model with different types of exploits and the critical phases.

2.3 New Technique - Context Conservation & Combo Technique

Guided by the model, we realize that context switches may influence the critical phases of kernel exploits. These prolonged critical phases may give competitor processes more chances to influence the kernel state when the exploits are performing critical actions, thus failing the exploits.

Guided by the insights from the model, we propose a new technique to detect and avoid context switches in kernel exploits to improve exploit reliability. Essentially, we run a timestamp reading logic in a loop, when a context switch happens, the logic will detect a large difference in timestamp compared with the previous iteration. We can then be sure that the exploit just enters a fresh time slice and can perform the critical actions without context switches. Our evaluation shows that this new technique can improve exploit reliability by 14.87%.

In a follow-up research, we noticed that context switches and interrupts are the reason why prefetch attacks are unreliable. In prefetch attacks, the exploit needs to prefetch a page many times to check whether the page exists or not. But if interrupts or context switches happen when the exploit is prefetching, a page may appear to

take long time to prefetch, which may be interpreted incorrectly as the page does not exist. By applying this context conservation technique, we can stabilize prefetch attacks and achieve 100% success rate in both physical and virtual machines.

The model also suggests that the unreliability factors are orthogonal to each other, which suggests that different stabilization techniques may be combined to better stabilize exploits. We applied all techniques that are applicable to each exploit and evaluated them. The result shows the combo technique leads to 135.53% improvement in exploit reliability, reaching a 90.94% success rate.

2.4 Conclusion

As evidenced by this research, by generalizing techniques used in known exploits, we can better understand a target system and discover new exploitation techniques.

3 Retspill

3.1 Introduction

Despite the advances in security mitigation, control flow hijacking attack still plays an important role in the exploitation of memory corruption vulnerabilities. Among the 70 exploits collected by Google's KernelCTF program, 52 are control flow hijacking attacks.

In the exploitation of the Linux kernel, attackers usually obtain control flow hijacking primitive (CFHP) through a vulnerability, then use creative techniques to bypass modern defenses, and finally compromise the whole system. The most important step in the process is to utilize user-controlled data injected into the kernel space using the CFHP to carry out advanced attacks, such as Return-Oritented-Programming (ROP).

In this work, we revisit the techniques that can turn CFHP into ROP, generalize them, identify another attack surface, and discover a new exploitation technique, named RetSpill, that can bypass all existing protections while having the least constraints on the required primitives.

3.2 Generalization - User-Controllable Data Injection

We first exhaustively collected 22 exploits, the largest dataset at the time of the project. Then we reviewed all existing works and identified all the techniques that could turn CFHP into ROP, as shown in Table 1.

As shown in the table, different techniques inject attacker-controlled data into different regions of the system with the same goal of accessing the controlled data using one gadget (the initial CHFP). This generalization of exploitation techniques inspired us to explore the possibility of data injection into other regions of the system.

Technique	Required Primitives	Region	Reliable*	Payload Rewrite	Still Applicable
ret2usr	CFHP	user space	✓	✓	✗
ret2dir [5]	CFHP+PL	physmap	✗‡	✓	✗
pivot to user	CFHP+CL	user space	✓	✓	✗
change CR4	CFHP+CL+rdi	user space	✗§	✓	✗
run_cmd	CFHP+CL+HL+rdi	kernel heap	✓	✓	✗△
pivot to heap	CFHP+CL+HL+regs	kernel heap	✓	✗	✓
KEPLER [10]	CFHP+CL+PL+regs	physmap	✗	✓	✓
set_memory_x	CFHP+CL+HL+rdi&rsi	kernel heap	✗§	✗	✓
RetSpill	CFHP+CL	kernel stack	✓	✓	✓

Table 1: With the development of modern protections in the Linux kernel, exploiting CFH vulnerabilities requires many primitives and becomes brittle. Our work, RetSpill, is a powerful exploitation technique that requires fewer primitives and yet is still reliable. * Given all the required primitives, whether the technique can derive the target primitive without failures. ‡ ret2dir is reliable only if given a page frame info leak primitive. § need to obtain CFHP twice. △ run_cmd function can get inlined, thus not exist, in some kernel builds. CL: code base info leak. HL: heal info leak. PL: physmap info leak.

Naturally, we turned our eyes to the kernel stack. A preliminary analysis showed that it is indeed possible to inject user-controlled data into the kernel stack. And due to the architecture design, we only need code base information leak to carry out the full exploitation, which is the least requirements among all techniques still applicable today.

3.3 New Technique - RetSpill

We named the new technique RetSpill. Essentially, it involves attackers injecting user-controlled data into the kernel stack, using the initial CFHP to shift the stack to the controlled data, and starting ROP. Due to the fact that the stack layout is deterministic, when the CFHP is obtained through a heap vulnerability, attackers

can invoke the CFHP many times, each with a different payload on the stack. It essentially gives attackers the ability to execute an infinite number of ROP chains.

One important factor that decides the effectiveness of RetSpill is the amount of data that users can control on the stack. To perform the evaluation comprehensively, we designed a system that could taint all user data when entering the kernel space through system calls, including user registers and memory that the kernel copies into kernel space. By running this system on the 22 exploits we collected, we observed that attackers could place 11 gadgets on the kernel stack on average, which is more than enough to carry out ROP attack to compromise the system. We further investigated the sources of the user data, identifying four sources where user data can be saved into the kernel stack: 1) Valid Data. Some user data is meant to be copied into the kernel stack by design. 2) Preserved Registers. User registers will be saved onto the kernel stack when the execution enters kernel space. 3) Calling Convention. Some registers or syscall arguments will be saved onto kernel stack due to the calling convention. 4) Uninitialized memory. A kernel thread always uses one region as the kernel stack. Attackers can inject data into the kernel stack using one system call, and when the target system call is invoked, some of the data will be left there because they are not initialized.

To demonstrate the severity of the new technique, we designed a system called igni, that can automatically turn a proof-of-concept (PoC) that obtains CFHP into a full-fledged privilege escalation (PE) exploit. We evaluated igni on the 22 exploits we collected, and it successfully generated 20 PE exploits.

3.4 Conclusion

By dissecting and generalizing known exploits, we can identify new attack surfaces and find new exploitation techniques as shown in this study.

4 ropbot

4.1 Introduction

Code reuse attack, such as Return-Oriented-Programming (ROP)[8], is a cornerstone in modern software security. Despite its early appearance in 2007, it still plays an important role in exploiting memory corruption vulnerabilities to this day. In the 52 control-flow-hijacking exploits collected by the KernelCTF program, 48 of them resolved to code reuse to compromise the system.

Generating code reuse payload, called CR chains (such as ROP chains), automatically, is a research challenge that many researchers have attempted in the past year. Despite its popularity, all tools in this field suffer from multiple limitations: most of them are only applicable to a limited set of architectures, such as x86/x64; and all of them rely on a bounded state exploration model to synthesize the payload, which fundamentally comes the dilemma of how to choose the proper bound.

In this work, we revisit the fundamentals of code reuse attacks, generalize their core mechanisms, and design a brand new technique for automatic CR chain generation. We present our research prototype, ropbot, that can automatically generate CR chains. Due to its generalized design, it is architecture-agnostic. We evaluated it on X64/MIPS/ARM/AARCH64/RISC-V and outperformed all existing works on their own datasets and benchmarks. Notably, it can generate 244 `execve("/bin/sh", 0, 0)` chains on a dataset of 272 binaries, more than double that of the second-best technique (90 chains).

4.2 Generalization - Gadget & rop_block

A gadget is a unit of reusable code snippet that serves as the basis of CR chains. Despite the fact that the idea has been around for more than 15 years, it is not

formally defined. As a result, the first challenge in generalizing code reuse attacks is: what is a gadget, especially outside the realm of X86/X64. Acknowledging the fact that a gadget is part of a CR chain that executes some logic and continues to execute another gadget, we define a gadget as a sequence of instructions that exists in the target binary, executes in order, and can *potentially* set the next PC to an attacker-controlled value.

Note that we relax the constraint of setting the PC to take conditional branches into account. In the language of symbolic execution, a gadget is defined as a code sequence that can move a symbolic value into the PC starting from a fully symbolized state (i.e. registers, stack, and memory).

We this definition, we are able to find all code snippets in a target binary that can potentially contribute to a CR chain. In our evaluation, we found a few new categories of gadgets. One example is that on ARM, code ending with `ldr pc, [sp], #<X>` is a gadget because it sets the next PC using values on the stack. Note that, under this definition, any code snippet containing an indirect jump is potentially a gadget. This is interesting as dynamic linking itself is built on top of indirect jumps using the Procedure Linkage Table (PLT) and the Global Offset Table (GOT). Invoking any function in the PLT (e.g., `call memset@plt`) will eventually lead to indirect jumps (e.g., `jmp qword ptr [memset@got]`). As a result, any code snippet ending with a PLT function call is a gadget, we call it GOT gadget. Another notable property of this definition is that gadgets can now span multiple basic blocks.

This generalized definition is architecture-agnostic, motivating us to find an architecture-agnostic CR chain generation solution. We revisited the CR payload from past exploits and noticed that gadgets are usually not used individually. Instead, they are used as a self-contained group to perform specific tasks. For example, `pop rax; ret` and `pop rbx; jmp rax` can be combined to gain the capability of `pop rbx`. Seeing

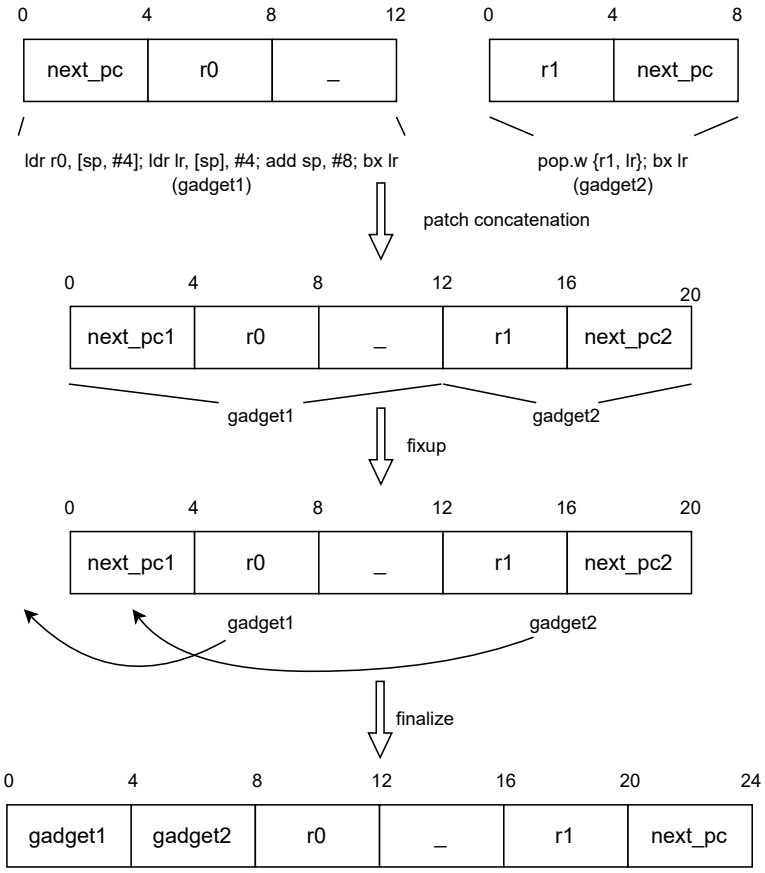

Figure 2: How to chain two rop_blocks together.

this, we define a generalized property called "self-contained". A gadget or a sequence of gadgets is called self-contained if and only if 1) it has a positive stack change (overall, it pops content from the stack); 2) it takes the next PC from the stack, and it is taken within the boundary of the stack change; 3) it contains no conditional branches. Then we call a self-contained group of gadgets a rop_block. For example, `pop rax; ret | pop rbx; jmp rax` is a rop_block.

4.3 New Technique - ropbot

Due to the definition, rop_blocks can be easily chained together as shown in Figure 2 And notice that the result of a rop_block chaining with another rop_block is also a rop_block, which essentially means a rop_block is a reusable piece in CR chaining.

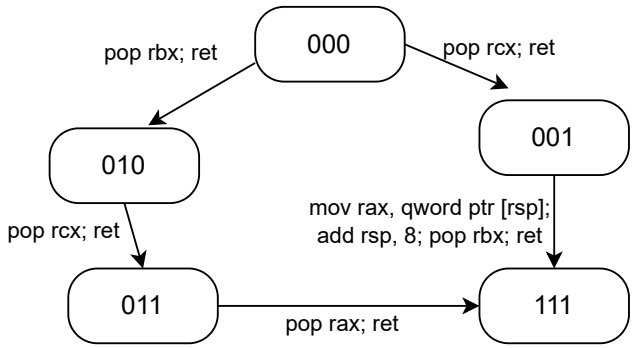

Figure 3: A graph of setting {rax, rbx, rcx}, where 001 means rax, rbx are not controlled while rcx is controlled.

Tool	facefeed Success	facefeed Time	execve Success	execve Time
ropbot	635	5.3h	244	0.7h
exrop[4]	357	23.4h	90	4.5h
crackers[6]	353	72.4h	0	22.1h
SGC[7]	616	130.9h	27	17.5h

Table 2: ropbot's evaluation result on generating the `0xfacefeed(0xdeadbeef, 0x40, 0x7b)` and `execve("/bin/sh", 0, 0)` chains.

So, we use each rop_block as a unit and perform effect analysis on them, such as register moving effect, register popping effect, concrete value effect (e.g. `xor rax, rax; set` essentially sets `rax` to 0).

When needing to generate a CR chain that sets registers, we can build a graph, where nodes represent register controls and edges are rop_block. As a result, the problem is transformed into finding a path from the node representing no register control to the node representing full register control. One example is shown in Figure 3.

We evaluated ropbot against state-of-the-art tools in the same field. And the preliminary result is shown in Table 2, which suggests that ropbot is not only the most capable code reuse chain generation tool but also the fastest. Our cross-arch

evaluation confirms that ropbot can work across architectures.

4.4 Conclusion

Revisiting and generalizing existing ideas in exploitation techniques can lead to the discovery of new and stronger techniques. As shown in this study, revisiting the idea of "gadget" leads to a more general definition, which inspired the use of rop_block. And the adoption of rop_block, which can be easily chained together, in turn leads to the discovery of graph-based exploit generation.

5 Plan For Defense

As part of the work of understanding and generalizing known software exploits to discover new exploitation techniques, I have published one of my works [11] that led to the discovery of the context conservation technique while analyzing the exploit reliability problem in existing kernel heap exploits.

Following this line of work, I published another work [12] that generalizes the issue of injecting user data into the kernel space and discovering the RetSpill exploitation technique.

As the final part of my thesis, I generalized the concept of gadget in automated code reuse attacks and proposed the robot system that can automatically generate code reuse payload across architectures and bypass the bounded state exploration dilemma. Currently, this work is under submission.

For the defense, I plan to improve ropbot and use a more comprehensive evaluation to show its contribution.

More specifically, I propose to do the following tasks for my defense:

- Better engineer ropbot and work towards open-sourcing it to benefit the community. Engineering errors in the system make it hard for us to evaluate the improvement contributed by the novelty. I plan to fix them before the final evaluation. I also plan to open-source ropbot so other researchers can directly build on top of it for further research.

- More robust evaluation. Currently, evaluation on ropbot is not comprehensive enough. For example, although ropbot supports x86 officially, we do not have a comprehensive evaluation for it. Another example is that we only evaluated 172 binaries for aarch64, which is not as comprehensive as on other architectures.

- More architecture support. Since one of the ropbot's contributions is being architecture-agnostic due to the generalization. I would like to extend the support for more architectures to demonstrate this ability. More specifically, I would like to extend the support to PowerPC, and potentially other architectures supported by vex.

REFERENCES

[1] Exploit database.

[2] kernelctf - google.

[3] What is lockbit ransomware — akamai.

[4] d4em0n. exrop.

[5] Vasileios P Kemerlis, Michalis Polychronakis, and Angelos D Keromytis. ret2dir: Rethinking kernel isolation. In *23rd USENIX Security Symposium (USENIX Security 14)*, pages 957–972, 2014.

[6] TF Melham and M DenHoed. Synthesis of code-reuse attacks from p-code programs. 2025.

[7] Moritz Schloegel, Tim Blazytko, Julius Basler, Fabian Hemmer, and Thorsten Holz. Towards automating code-reuse attacks using synthesized gadget chains. In *Computer Security–ESORICS 2021: 26th European Symposium on Research in Computer Security, Darmstadt, Germany, October 4–8, 2021, Proceedings, Part I 26*, pages 218–239. Springer, 2021.

[8] Hovav Shacham. The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86). In *Proceedings of the 14th ACM conference on Computer and communications security*, pages 552–561, 2007.

[9] Contributors to Malware Database Wikia. Wannacry ransomware — malware database wikia — fandom.

[10] Wei Wu, Yueqi Chen, Xinyu Xing, and Wei Zou. {KEPLER}: Facilitating control-flow hijacking primitive evaluation for linux kernel vulnerabilities. In *28th USENIX security symposium (USENIX security 19)*, pages 1187–1204, 2019.

[11] Kyle Zeng, Yueqi Chen, Haehyun Cho, Xinyu Xing, Adam Doupé, Yan Shoshitaishvili, and Tiffany Bao. Playing for {K (H) eaps}: Understanding and improving linux kernel exploit reliability. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 71–88, 2022.

[12] Kyle Zeng, Zhenpeng Lin, Kangjie Lu, Xinyu Xing, Ruoyu Wang, Adam Doupé, Yan Shoshitaishvili, and Tiffany Bao. Retspill: Igniting user-controlled data to burn away linux kernel protections. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 3093–3107, 2023.